OS Lab Companion

Operating Systems Lab project report · 12 October 2026
Anuj Deshpande · Registration No.: 24BCE0794

We built a local study tool around the material supplied for our Operating Systems course. It searches the notes, shows the file and page or slide behind each result, and can ask a local language model to explain the retrieved text. It also reviews selected C lab-code patterns without running student code and has a separate CPU scheduling simulator.

1. Dataset

The main input was the course `OS.zip` archive. It contained 54 non-directory entries. We extracted searchable text from 33 files: 16 PPTX, 15 PDF and 2 DOCX. This produced 553 passages. We kept the original filename and page or slide for each passage. The archive and processed text stay on the local machine and are excluded from the public Git repository.

The importer skipped 21 entries: 19 were formats this version cannot extract as text, mainly older PowerPoint files and images; two were temporary or project-planning files. A separate public demo set has three short Markdown lab notes, split into 21 instructional sections, and five reviewed C cases. We use that small set for repeatable tests. It is not a substitute for the course archive.

We did not train or fine-tune the language model on these files. The application retrieves relevant passages at question time and sends only those passages, plus any pasted code, to a model running on this computer.

2. Preprocessing

The ZIP importer rejects unsafe paths and oversized entries, reads supported files, and records a reason for every skipped entry. PDF text is extracted page by page, PPTX text slide by slide, and DOCX text by paragraph. We normalize Unicode and spacing, divide long text into chunks of up to 210 words, and discard chunks shorter than 60 characters. A SHA-256 hash identifies each source file; a second stable ID identifies each passage.

The resulting local JSON index stores the text, file type, title, original location and hash. At search time we use BM25 for word matching and latent semantic analysis (TF-IDF followed by truncated SVD) for statistical similarity. Reciprocal-rank fusion combines their rankings. LSA is a local low-rank text method, not a neural embedding model.

```
Course ZIP → page/slide extraction → text cleanup → passages with source locations → BM25 +
LSA → ranked evidence
```

The app exposes the per-file passage counts, hashes and skip reasons so that an import can be checked instead of treating the ZIP as a black box.

3. Code implementation

The backend is Python and serves a local web interface. `oslab/course.py` performs import and hybrid retrieval. `oslab/code_review.py` finds selected OS calls such as `fork`, `pipe`, `read`, `wait`, mutex and semaphore calls; it reports source lines and cautious next checks. `oslab/local_model.py` calls a local Qwen3 4B instruction model through llama.cpp. The model returns structured answer points and visual steps with source IDs. The backend rejects invalid source IDs and suppresses an answer when the retrieved passages do not cover the question.

For a pipe reader looping until EOF with no `close()` call, the code reviewer produces a checked walkthrough from the source lines and pipe notes. This case is handled directly because our model tests showed it could invent descriptor operations. Other code questions may receive model suggestions, always labelled as hypotheses. Pasted code is never compiled or executed. The scheduling lab is separate Python logic for FCFS, nonpreemptive SJF and Round Robin; its timelines and waiting-time values are calculated, not generated by the model.

```
Question → retrieve passages → local explanation + cited visual
Code → static clues + retrieve passages → checked walkthrough or model suggestion
Workload → deterministic scheduling simulator → event timeline and metrics
```

oslab/course.py · actual source, lines 95–112

```
                    content = _clean(raw_text)
                    for part, chunk in enumerate(section_chunks(content, max_words=210), 1):
                        chunk = _clean(chunk)
                        if len(chunk) < 60:
                            continue
                        record_id = hashlib.sha256(f"{name}|{location}|{part}|{chunk}".encode()).hexdigest()[:16]
                        records.append({"id": record_id, "source": name, "title": path.stem,
                                        "group": group, "format": suffix[1:], "location": location,
                                        "part": part, "text": chunk})
                documents.append({"source": name, "format": suffix[1:], "chunks": len(records) - before,
                                  "sha256": digest})
            except Exception as error:
                skipped.append({"source": name, "reason": f"extraction failed: {type(error).__name__}"})
    if not records:
        raise ValueError("No searchable text could be extracted from this archive")
    payload = {"schema_version": 1, "documents": documents, "skipped": skipped, "records": records}
    index_path.parent.mkdir(parents=True, exist_ok=True)
    index_path.write_text(json.dumps(payload, ensure_ascii=False), encoding="utf-8")
```

Figure 1. Actual lines from `oslab/course.py`. The importer creates passage IDs, preserves source location and records file hashes and extraction failures. Full source is in the repository.

The local model is optional for basic retrieval, the checked pipe example and scheduling. The UI shows the original excerpts even when the model is unavailable.

4. Metrics, results and discussion

The private course import gave 33 readable files and 553 passages from 54 entries. That is an extraction count, not a measure of answer accuracy. We have not yet labelled relevance judgments for the private course material, so we cannot report Recall@k or model accuracy on that archive.

For the small public demo set, we reran the repository evaluation. The parser found all 21 labelled Markdown headings across three files, with 1.000 heading precision and recall and complete recorded metadata. This only checks the controlled Markdown format; it says nothing about OCR or PDF extraction quality.

Retrieval method	Recall@3	MRR@10	nDCG@10
BM25	0.917	0.701	0.777
LSA	0.833	0.722	0.791
Hybrid rank fusion	1.000	0.847	0.886

Table 1. Twelve labelled queries over the small public demo corpus, one relevant section per query. These values must not be reported as performance on the professor's files.

The earlier fork/wait diagnostic benchmark matched all four held-out failure variants at rank 1. Two variants share each failure label, so 4/4 is a regression check for that narrow setup, not evidence that arbitrary student programs can be debugged. No Linux runtime traces were collected for this benchmark.

Manual checks of the current course search found relevant passages for page faults and semaphores. A request comparing disk-scheduling algorithms was a useful failure case: the ZIP contained a slide naming disk scheduling but not enough detail for that comparison. The app now says the evidence is insufficient and leaves the closest passages visible. The browser smoke check passed the theory, code review, scheduling and data views. None of these checks is a student learning study.

Reproduce the controlled metrics with `python -m oslab evaluate`. The values above come from `results/processing_metrics.json`, `results/retrieval_metrics.json` and `results/diagnostic_metrics.json`.

5. What went wrong

When we first tried the app with a pasted C program, search found a useful note about pipes. That felt like progress, but it still left the student asking the main question: why does my program hang? The note explained the idea in general; it could not connect it to the loop and missing `close()` call in that program. This pushed us to add a local explanation model and a walkthrough tied to the code.

The model made the answer easier to read, but it created a new problem. In one test, it confidently said the parent had closed a pipe end, although there was no `close()` in the pasted code. We had to check its explanation against the actual lines and write a line-by-line walkthrough for that case. A clear explanation can still be wrong.

We also thought that importing the course ZIP meant we could search the whole syllabus. Some files were older presentations or images that our importer could not read. A search result could look relevant while only mentioning a topic in passing. We now show skipped files and avoid giving a confident answer when the material does not support one.

Our early test scores looked reassuring, but they came from a small set of questions we wrote ourselves. They show that the demo cases work; they do not show how well the tool answers questions across the professor's course material. We should have tested real course questions against those files sooner.

---

6. Alternative flow and improvement

If we repeated the project, we would start with the course archive rather than the first lab example. We would list file types, convert the older presentations, add OCR for scans, and sample extracted pages by hand. We would keep a small import log of files that failed and why. That would make coverage gaps visible before building the teaching interface.

Next we would ask a faculty member or students to label about 30–50 real course questions with relevant pages. Those labels would let us compare BM25, LSA and hybrid search on the same archive the app uses. We would inspect failed queries by topic and distinguish “not in the archive” from “present but ranked badly.” This is the missing evaluation behind the current 553-passage count.

Only after retrieval passed that check would we add the local model. We would test whether each generated claim is actually supported by its cited passage, and record refusal cases separately from wrong answers. For questions that require calculation or state changes, we would prefer a tested program, as we did for CPU scheduling. For other topics, the model-made visual would remain an explanation rather than a claimed execution trace.

For lab-code review, the next stage would use a parser or compiler-assisted analysis to understand control flow and descriptor ownership. A carefully isolated runner for approved examples could then collect real outputs or traces. We would display source inference, expected behavior and observed behavior in separate fields. Until then, the current reviewer should be presented as a source-based study aid, not a general debugger.

This order would have made the project easier to explain: a checked course dataset first, measured retrieval second, then local explanations and tools added only where they answer a demonstrated student need.

The report describes the build as tested on 12 October 2026. Course files and model weights are local and are not part of the public source repository.